03-02 Fetch Stage 구현 분석

개요

Fetch Stage는 TPU 파이프라인의 첫 번째 단계로, 네트워크에서 트랜잭션 패킷을 수신하고 이를 다음 단계인 SigVerify Stage로 전달하는 역할을 담당합니다. UDP 소켓과 QUIC 프로토콜을 통해 다양한 유형의 트랜잭션을 효율적으로 수신합니다.

FetchStage 구조체 정의

```
pub struct FetchStage {
thread_hdls: Vec<JoinHandle<()>>,
}
```

FetchStage는 여러 개의 스레드 핸들을 관리하는 단순한 구조체입니다. 각 스레드는 서로 다른 소켓에서 패킷을 수신하는 역할을 담당합니다.

소켓 유형별 처리

1. 트랜잭션 소켓 (TPU Sockets)

```
let tpu_threads: Vec<_> = if tpu_enable_udp {
tpu_sockets
.into_iter()
.enumerate()
```

```
.map(|(i, socket)| {
streamer::receiver(
format!("solRcvrTpu{i:02}"),
socket,
exit.clone(),
sender.clone(),
recycler.clone(),
tpu_stats.clone(),
coalesce,
true,
in_vote_only_mode.clone(),
)
})
.collect()
} else {
Vec::default()
};
```

일반 트랜잭션을 수신하는 소켓들입니다:

- **스레드 이름**: `solRcvrTpu{i:02}` 형식
- **목적**: 클라이언트로부터 직접 전송된 트랜잭션 수신
- **출력**: `sender` 채널로 패킷 전달

2. 전달 트랜잭션 소켓 (TPU Forwards Sockets)

```
let  tpu_forwards_threads: Vec<_> = if  tpu_enable_udp {
tpu_forwards_sockets
.into_iter()
.enumerate()
.map(|(i, socket)| {
streamer::receiver(
format!("solRcvrTpuFwd{i:02}"),
socket,
exit.clone(),
forward_sender.clone(),
recycler.clone(),
tpu_forward_stats.clone(),
coalesce,
true,
in_vote_only_mode.clone(),
)
})
.collect()
} else {
Vec::default()
};
```

다른 검증자로부터 전달받은 트랜잭션을 수신하는 소켓들입니다:

- **스레드 이름**: `solRcvrTpuFwd{i:02}` 형식
- **목적**: 다른 검증자가 전달한 트랜잭션 수신
- **출력**: `forward_sender` 채널로 패킷 전달

3. 투표 소켓 (TPU Vote Sockets)

```
let  tpu_vote_threads: Vec<_> = tpu_vote_sockets
.into_iter()
.enumerate()
.map(|(i, socket)| {
streamer::receiver(
format!("solRcvrTpuVot{i:02}"),
socket,
exit.clone(),
vote_sender.clone(),
recycler.clone(),
tpu_vote_stats.clone(),
coalesce,
true,
None,
)
})
.collect();
```

투표 트랜잭션을 수신하는 소켓들입니다:

- **스레드 이름**: `solRcvrTpuVot{i:02}` 형식
- **목적**: 합의 과정에서 사용되는 투표 트랜잭션 수신
- **출력**: `vote_sender` 채널로 패킷 전달
- **특징**: `in_vote_only_mode` 제한 없음

전달 패킷 처리 메커니즘

handle_forwarded_packets() 함수

```
fn handle_forwarded_packets(
recvr: &PacketBatchReceiver,
sendr: &PacketBatchSender,
poh_recorder: &Arc<RwLock<PohRecorder>>,
) -> Result<()> {
let mark_forwarded = |packet: &mut Packet| {
packet.meta_mut().flags |= PacketFlags::FORWARDED;
};

let mut packet_batch = recvr.recv()?;
let mut num_packets = packet_batch.len();
packet_batch.iter_mut().for_each(mark_forwarded);
let mut packet_batches = vec![packet_batch];
while let Ok(mut packet_batch) = recvr.try_recv() {
```

```
packet_batch.iter_mut().for_each(mark_forwarded);

num_packets += packet_batch.len();

packet_batches.push(packet_batch);

// Read at most 1K transactions in a loop

if  num_packets > 1024 {

break;

}

}



if  poh_recorder

.read()

.unwrap()

.would_be_leader(HOLD_TRANSACTIONS_SLOT_OFFSET.saturating_mul(DEFAULT_TICKS_PER_

{

inc_new_counter_debug!("fetch_stage-honor_forwards", num_packets);

for  packet_batch  in  packet_batches {

if  sendr.send(packet_batch).is_err() {

return Err(Error::Send);

}

}

} else {

inc_new_counter_info!("fetch_stage-discard_forwards", num_packets);

}
```

```
Ok(())
}
```

주요 기능:

1. **패킷 마킹**: 전달된 패킷에 `FORWARDED` 플래그 설정
2. **배치 처리**: 최대 1024개 패킷까지 한 번에 처리
3. **리더십 확인**: PoH Recorder를 통해 리더 자격 확인
4. **조건부 전달**: 리더일 때만 패킷을 다음 단계로 전달

패킷 수신 메커니즘 (Streamer 모듈)

recv_loop() 함수

```
fn recv_loop(
socket: &UdpSocket,
exit: &AtomicBool,
packet_batch_sender: &PacketBatchSender,
recycler: &PacketBatchRecycler,
stats: &StreamerReceiveStats,
coalesce: Duration,
use_pinned_memory: bool,
in_vote_only_mode: Option<Arc<AtomicBool>>,
) -> Result<()> {
```

```
loop {
let  mut  packet_batch = if  use_pinned_memory {
PacketBatch::new_with_recycler(recycler, PACKETS_PER_BATCH, stats.name)
} else {
PacketBatch::with_capacity(PACKETS_PER_BATCH)
};
loop {
// Check for exit signal
if  exit.load(Ordering::Relaxed) {
return Ok(());
}


// Check vote-only mode
if  let Some(ref  in_vote_only_mode) = in_vote_only_mode {
if  in_vote_only_mode.load(Ordering::Relaxed) {
sleep(Duration::from_millis(1));
continue;
}
}


if  let Ok(len) = packet::recv_from(&mut  packet_batch, socket, coalesce) {
if  len > 0 {
// Update statistics
```

```
stats.packets_count.fetch_add(len, Ordering::Relaxed);

stats.packet_batches_count.fetch_add(1, Ordering::Relaxed);

stats.max_channel_len.fetch_max(packet_batch_sender.len(), Ordering::Relaxed);

if  len == PACKETS_PER_BATCH {

stats.full_packet_batches_count.fetch_add(1, Ordering::Relaxed);

}



packet_batch_sender.send(packet_batch)?;

}

break;

}

}

}

}
```

핵심 특징:

1. **메모리 재활용**: PacketBatchRecycler를 사용한 효율적인 메모리 관리
2. **배치 처리**: 여러 패킷을 하나의 배치로 묶어서 처리
3. **통계 수집**: 실시간 성능 모니터링을 위한 통계 정보 수집
4. **투표 전용 모드**: 네트워크 혼잡 시 투표 트랜잭션만 처리하는 모드

성능 최적화 기법

1. 패킷 병합 (Coalescing)

```
if let Ok(len) = packet::recv_from(&mut packet_batch, socket, coalesce) {
// 패킷 처리
}
```

- **coalesce** 매개변수를 통해 여러 패킷을 하나의 배치로 병합
- 시스템 호출 횟수 감소로 성능 향상

2. 메모리 재활용

```
let recycler: PacketBatchRecycler = Recycler::warmed(1000, 1024);
```

- 미리 할당된 메모리 풀 사용
- 가비지 컬렉션 압력 감소
- 메모리 할당/해제 오버헤드 최소화

3. 다중 소켓 처리

```
let tpu_threads: Vec<_> = tpu_sockets
.into_iter()
.enumerate()
.map(|(i, socket)| {
// 각 소켓마다 별도 스레드 생성
})
```

```
.collect();
```

- 여러 UDP 소켓을 병렬로 처리
- 네트워크 대역폭 최대 활용
- 단일 소켓 병목 현상 방지

통계 및 모니터링

StreamerReceiveStats 구조체

```rust
pub struct StreamerReceiveStats {
pub name: &'static str,
pub packets_count: AtomicUsize,
pub packet_batches_count: AtomicUsize,
pub full_packet_batches_count: AtomicUsize,
pub max_channel_len: AtomicUsize,
}
```

수집되는 메트릭:

- **packets_count**: 수신된 총 패킷 수
- **packet_batches_count**: 처리된 패킷 배치 수
- **full_packet_batches_count**: 최대 크기 배치 수
- **max_channel_len**: 채널 큐 최대 길이

메트릭 보고

```
let metrics_thread_hdl = Builder::new()
.name("solFetchStgMetr".to_string())
.spawn(move || loop {
sleep(Duration::from_secs(1));


tpu_stats.report();
tpu_vote_stats.report();
tpu_forward_stats.report();


if exit.load(Ordering::Relaxed) {
return;
}
})
.unwrap();
```

1초마다 통계 정보를 보고하는 전용 스레드가 실행됩니다.

오류 처리

주요 오류 유형

```
match e {
Error::RecvTimeout(RecvTimeoutError::Disconnected) => break,
Error::RecvTimeout(RecvTimeoutError::Timeout) => (),
```

```
Error::Recv(_) => break,

Error::Send => break,

_ => error!("{:?}", e),

}
```

- **Disconnected**: 채널 연결 해제 시 스레드 종료
- **Timeout**: 타임아웃은 정상적인 상황으로 처리
- **Send**: 다음 단계로 전달 실패 시 스레드 종료

요약

Fetch Stage는 Solana TPU의 관문 역할을 하며, 다음과 같은 핵심 기능을 제공합니다:

1. **다중 소켓 관리**: UDP와 QUIC 프로토콜을 통한 패킷 수신
2. **효율적인 배치 처리**: 메모리 재활용과 패킷 병합을 통한 성능 최적화
3. **트래픽 분류**: 일반 트랜잭션, 전달 트랜잭션, 투표 트랜잭션 구분 처리
4. **리더십 기반 필터링**: PoH Recorder를 통한 조건부 패킷 처리
5. **실시간 모니터링**: 성능 메트릭 수집 및 보고

이러한 설계를 통해 Fetch Stage는 높은 처리량과 낮은 지연시간을 달성하면서도 네트워크 혼잡 상황에서의 안정성을 보장합니다.